

기술 아키텍처

1. 실시간 미디어 전송

■ 미디어 서버

- 사용 기술: LiveKit (WebRTC SFU)
- 선택 이유
 - 교수 음성을 백엔드 프록시 없이 직접 publish/subscribe 모델로 전달해야 하므로, 중간 서버 없는 실시간 스트리밍이 필수
 - AI 워커가 participant로 room에 참여해 오디오 track을 subscribe하고, 번역 결과를 새 track 으로 publish 하는 구조가 자연스러움
 - 언어별 오디오 track(tts.zh-CN, tts.vi-VN 등)과 자막용 text stream 을 분리 관리해야 하는데, LiveKit 의 track/data channel 구조가 이에 정확히 맞음
- 장점
 - Room → Participant → Track 계층 구조로 교수/학생/AI 워커 역할 분리가 명확
 - 학생은 자기 언어의 track만 subscribe하면 되므로 클라이언트 로직이 단순
 - WebRTC 기반이라 지연이 수백 ms 이내로 낮음
 - 오디오·비디오·데이터를 하나의 room에서 통합 관리
 - text stream(DataChannel) 지원 → 자막 전송에 별도 WebSocket 불필요
- 비교
 - LiveKit Cloud 또는 self-hosted 선택 가능
 - SDK: Flutter(교수 앱), Flutter(학생 앱), Python(AI 워커) 모두 제공

2. 음성 인식 (STT)

■ STT

- 사용 기술: Azure Speech Real-time STT
- 선택 이유
 - 교수 발화를 실시간으로 텍스트 변환해야 하므로, streaming 방식의 real-time transcription이 필수

- 전공 용어·약어 인식률을 높이기 위해 phrase list와 custom speech 기능이 필요
- ko-KR locale 지원이 안정적이어야 함
- 장점
 - Real-time transcription: partial(중간 결과) + final(확정 결과)을 실시간으로 반환 → 자연스러운 자막 흐름 구현 가능
 - Phrase list: 수업 시작 시 해당 과목의 전공 용어 목록을 주입하면 인식 정확도 향상
 - Custom Speech: 학과 단위로 자체 학습 데이터를 업로드해 모델 미세조정 가능 → 반복적으로 쓰이는 학과 용어에 특히 유효
 - Language detection 지원 → 향후 다국어 교수 대응 확장 가능
 - Speech SDK로 Python에서 비동기 스트리밍 처리 가능
- 비교
 - locale 기본값: ko-KR
 - phrase list 는 세션 시작 시 glossary에서 자동 주입
 - custom speech는 MVP 이후 학과별 적용 검토

3. 검색 / RAG

■ RAG 검색 엔진

- 사용 기술: Azure AI Search (Hybrid Search)
- 선택 이유
 - 강의자료 기반 문맥 보정이 필요하지만, 매 문장마다 검색하면 지연이 커지므로 조건부 검색(RAG Trigger) 방식 채택
 - 전공 용어는 키워드 매칭이 정확하고, 문맥 유사도는 벡터 검색이 강하므로 둘을 동시에 쓸 수 있는 hybrid search 가 필요
- 장점
 - Hybrid Search: keyword(BM25) + vector(HNSW)를 한 요청에서 병렬 실행, RRF(Reciprocal Rank Fusion)로 결과 병합
 - Semantic Ranking: 검색 결과에 추가로 의미 기반 재정렬 적용 가능 → 강의 문맥에 더 적합한 chunk 상위 노출
 - 필터 필드 지원 → 과목별, 주차별, 자료 유형별 검색 범위 제한 가능

- 관리형 서비스이므로 인덱스 관리·스케일링을 직접 하지 않아도 됨
- 비교
 - MVP 로컬 실험용으로 FAISS 사용 가능하나, 운영 기준은 Azure AI Search
 - 검색 우선순위: ① 이번 주 교수 업로드 자료 → ② 과목 보조자료 → ③ 학과 공통 지식팩 → ④ 전공서적/교재
 - RAG Trigger 조건: glossary 용어 등장, 약어/기호, STT 불확실성, 다의어, 문맥 불안정 문장

번역 (Translation LLM)

▮ 번역 엔진

- 사용 기술: OpenAI Responses API + gpt-5.4-mini
- 선택 이유
 - 실시간 강의 번역은 고빈도·저지연 요청이므로, 빠른 응답 속도의 mini 모델이 적합
 - 언어별 번역 결과를 안정적인 JSON 구조로 받아야 후속 TTS 파이프라인과 연결이 깔끔함
 - OpenAI가 신규 프로젝트에 Responses API를 권장하고 있음
- 장점
 - Structured Outputs (JSON Schema): 언어별 번역 결과를 스키마에 맞춰 강제 출력 → 파싱 실패 없음
 - gpt-5.4-mini: 고빈도 워크로드 최적화 mini 모델 → 비용 대비 속도 우수
 - 시스템 프롬프트에 glossary를 주입해 전공 용어 번역 일관성 확보
 - 한 번의 API 호출로 다국어 동시 번역 가능 (JSON 내 언어별 필드)
- 비교
 - RAG context가 있으면 프롬프트에 포함, 없으면 context 없이 바로 번역
 - 세션×언어별 1회 호출 (학생별 호출 아님)

음성 합성 (TTS)

▮ TTS

- 사용 기술: Azure Speech TTS (Neural Voice)
- 선택 이유

- STT와 동일한 Azure Speech 플랫폼을 사용해 음성 입출력을 통일 → 관리 포인트 축소
- 전공 용어 발음을 제어해야 하므로 SSML, phoneme, custom lexicon 기능이 필요
- 다국어 locale별 자연스러운 neural voice가 필요
- 장점
 - Neural Voice: locale 별로 자연스러운 음성 제공 (zh-CN, vi-VN, mn-MN 등)
 - SSML 지원: pitch, rate, voice style 세밀 조절 가능
 - Phoneme / Custom Lexicon: glossary 기반으로 전공 용어 발음을 명시적으로 지정 가능
 - 스트리밍 합성 지원 → 생성과 동시에 LiveKit track으로 전송 가능
- 비교
 - TTS는 세션×언어별 1회 생성 → 같은 언어 학생 30명이 있어도 음성 생성은 1번
 - glossary 의 발음 정보를 STT phrase list와 TTS lexicon 에 공통 적용

6. 교수용 앱 (프론트엔드)

■ 교수 웹

- 사용 기술: Flutter + LiveKit Flutter SDK
- 선택 이유
 - 교수가 브라우저에서 바로 마이크를 켜고 LiveKit room에 음성을 publish해야 함
 - 강의자료 업로드, 수업 시작/종료, QR 생성 등 관리 기능도 필요
- 장점
 - React 생태계의 풍부한 UI 컴포넌트 활용
 - LiveKit JS SDK 로 WebRTC 연결을 간단히 구현
 - SSR/SSG로 초기 로딩 최적화
 - Next.js API Routes 로 간단한 서버 로직 처리 가능

7. 학생용 앱 (프론트엔드)

■ 학생 앱

- 사용 기술: Flutter + LiveKit Flutter SDK

- 선택 이유
 - iOS/Android 동시 지원 필요 (유학생 기기 다양)
 - 이어폰으로 실시간 오디오를 듣는 것이 주 경험이므로 네이티브 오디오 제어가 중요
- 장점
 - 단일 코드베이스로 iOS/Android 동시 배포
 - LiveKit Flutter SDK 로 track subscribe/재생 구현
 - 언어 선택 → 해당 locale track만 subscribe하는 로직이 단순
 - 자막 표시를 위한 text stream 수신도 동일 SDK 내에서 처리

1. 제어 API (백엔드)

■ 제어 API

사용 기술: NestJS (Node.js)

선택 이유

- 학교/과목/교수/세션 관리, 권한/토큰 발급, QR 생성, 자료 업로드 등 CRUD 중심

API

- 음성 처리와 분리하여 웹 개발 역량으로 빠르게 구현 가능

장점

- TypeScript 기반 구조화된 모듈 시스템 → 도메인별 모듈 분리 용이
- Guard/Interceptor 로 인증·권한 로직 깔끔하게 처리
- LiveKit Server SDK (Node.js)로 room token 발급
- Swagger 자동 생성으로 API 문서화

비고

- 음성 프록시 역할은 하지 않음 — 세션·권한·메타데이터 관리만 담당

2. 실시간 AI 워커

■ AI 워커

사용 기술: Python (FastAPI / asyncio)

선택 이유

- STT → Segmenter → RAG Trigger → Retriever → Translator → TTS → Publisher

파이프라인을 비동기로 처리해야 함

- Azure Speech SDK, OpenAI SDK, Azure AI Search SDK 모두 Python 지원이 가장 성숙

장점

- asyncio 기반 비동기 처리로 I/O 대기 시간 최소화
- AI/ML 라이브러리 생태계 최고 수준

- 각 외부 API SDK의 Python 지원이 가장 빠르고 안정적
- LiveKit Python SDK 로 room participant 로 참여, track publish/subscribe 가능
- 세션당 1개 워커 할당 (세션 전담)
- 수업 시작 시 glossary, 자료 chunk를 Redis에서 prewarm

3. 데이터 레이어

■ DB

사용 기술: PostgreSQL

선택 이유

- 학교·학과·과목·교수·세션·glossary 등 관계형 데이터 관리

장점

- ACID 트랜잭션, JSON 컬럼 지원, 확장성, 무료 오픈소스

■ 파일 저장소

사용 기술: Azure Blob Storage

선택 이유

- 교수 업로드 PDF/PPT 원본 및 전처리 결과 저장

장점

- 대용량 파일 저장에 최적화, Azure 생태계 내 통합 용이, CDN 연동 가능

■ 캐시

사용 기술: Redis

선택 이유

- 세션 상태, prewarm glossary, 최근 segment window 등 실시간 데이터의 빠른

읽기/쓰기

장점

- 인메모리 기반 초저지연 (sub-ms)
- TTL 설정으로 세션 종료 시 자동 정리
- Pub/Sub으로 워커 간 이벤트 전달 가능

[UniVoice]전공 특화 RAG를 활용한 실시간 강의 번역(MVP)

1. 서비스 개요

본 서비스는 한국 대학에서 강의를 수강하는 외국인 유학생의 언어 장벽 문제를 해결하기 위해 한국어 강의를 실시간으로 번역하여 음성 및 자막으로 제공하는 시스템이다.

- 교수의 강의 음성을 실시간으로 수집
- → 강의 자료 기반으로 문맥을 보정

- → 유학생의 언어로 번역 후 음성 출력

이를 통해 유학생은 강의를 현지 학생과 같은 속도로 이해할 수 있다.

2. 간략 시스템 아키텍처

전체 흐름

1. 강의안 입력 (교수)

- PDF / PPT 업로드
- 텍스트 추출 및 임베딩 생성
- FAISS 벡터 DB에 저장 (강의자료 RAG)

2. 음성 전달

- LiveKit 기반 실시간 오디오 스트리밍
- Audio Gateway로 안정적 연결

3. STT

- Azure Speech Real-time STT

4. Semantic Router

- 입력 문장을 분류:
 - 일반 발화
 - 전공/강의 관련 발화
- (지연 시간 감소 방안) 적용 시 병렬로 RAG와 함께 처리하다가 semantic router의 결과에 따라 RAG 과정을 중단하는 방향으로 사용

5. 쿼리

- 입력 쿼리(강의 STT) → 검색 쿼리(RAG 검색에 맞게 정제된 텍스트) → 최종 llm 입력(검색 결과까지 붙은 프롬프트)

6. DualRAG

- 전공 RAG (교재 기반)
 - 강의자료 RAG (업로드 자료 기반)
- 두 RAG를 결합하여 문맥 보정
- 방식 3가지

1. 추론 시점에서 통합하기(RAG 2개 다 살려서 추론에 사용)

두 결과를 재정렬/병합 후 llm에 전달

장점

- 전공과 강의 특화 문맥 유지가 더 잘 됨
- 검색 가중치 조절 쉬움
 - rule 추가 쉬움
- router와 잘 맞음

단점

- 검색 2번 = 소요 시간 증가(ms 단위라고 해서 큰 병목은 아닐 가능성이 높다고 함)
 - 병렬로 사용해야 함

2. 하나의 벡터 DB로 합치되 메타데이터로 source를 유지

source : major(전공), lecture(강의자료)

물리적으로는 1개인데 논리적으로는 2개

장점

- 하나의 DB로 운영 편의성 확보
- 필요하면 source 필터나 가중치 적용 가능
- 추론 시 source별 top-k도 가능

3. 완전히 섞어서 source 구분 없이 사용

- 전공 지식이 강의자료보다 더 많이 뽑히거나 슬라이드 단편이 기본 개념보다 우선될 수 있다.
- 검색 실패 원인 분석 어려움

7. 번역 LLM

- OpenAI Realtime API (빠른 MVP 구현)
- (대안: Gemma 3 4B 경량화)

8. TTS

- Google TTS / Azure TTS (MVP 구현하며 결정: 속도와 확정 용이성 중 택)

9. 음성 전달

- WebSocket(빠른 MVP 구현)

- **MVP**

- Azure STT → RAG → LLM → TTS까지는 서버에서 처리
- 서버가 생성된 TTS 오디오를 WebSocket으로 학생 앱에 스트리밍
- 앱은 받은 청크를 바로 재생
- 같은 연결로 자막, 에러, "번역 중" 같은 이벤트도 같이 전달

- **실서비스**

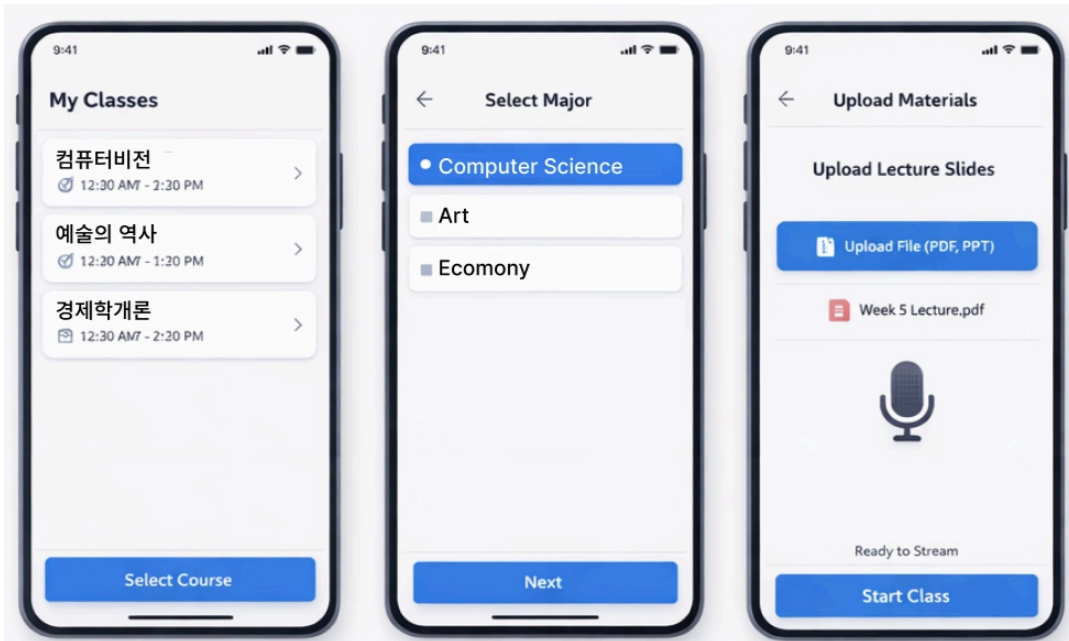
- 학생 앱도 LiveKit/WebRTC로 TTS 오디오 트랙을 구독
- 서버는 생성된 TTS를 오디오 트랙으로 publish
- 학생은 자기 언어 채널만 subscribe

10. 전송 및 출력

- 유학생의 앱으로 음성 스트리밍
- 선택적으로 자막 제공
 - 보조적인 도구로써 강의 흐름 이해에 중점을 둔 요약 자막
 - 번역 결과를 그대로 보여주는 자막
- 1:N 동시 다국어 출력
 - 공통 처리: Azure STT → Router → RAG → base context
 - STT 결과 + RAG로 먼저 **의미가 정리된 중간 표현**
 - 또는 **용어 교정된 한국어 문장**
 - 또는 **간단한 의미 단위 표현**
 - 언어별 fan-out & 언어별 큐 운영
 - e.g. 영어 큐, 중국어 큐, 베트남어 큐, ...
 - 같은 언어 고른 학생들은 결과 공유 가능
 - 큐 길이나 부하가 커지면 **워커 인스턴스 자동 확장**
 - 워커 : 큐에 쌓인 일을 실제로 처리하는 처리기
 - 자주 쓰는 언어는 워커를 더 많이 둬

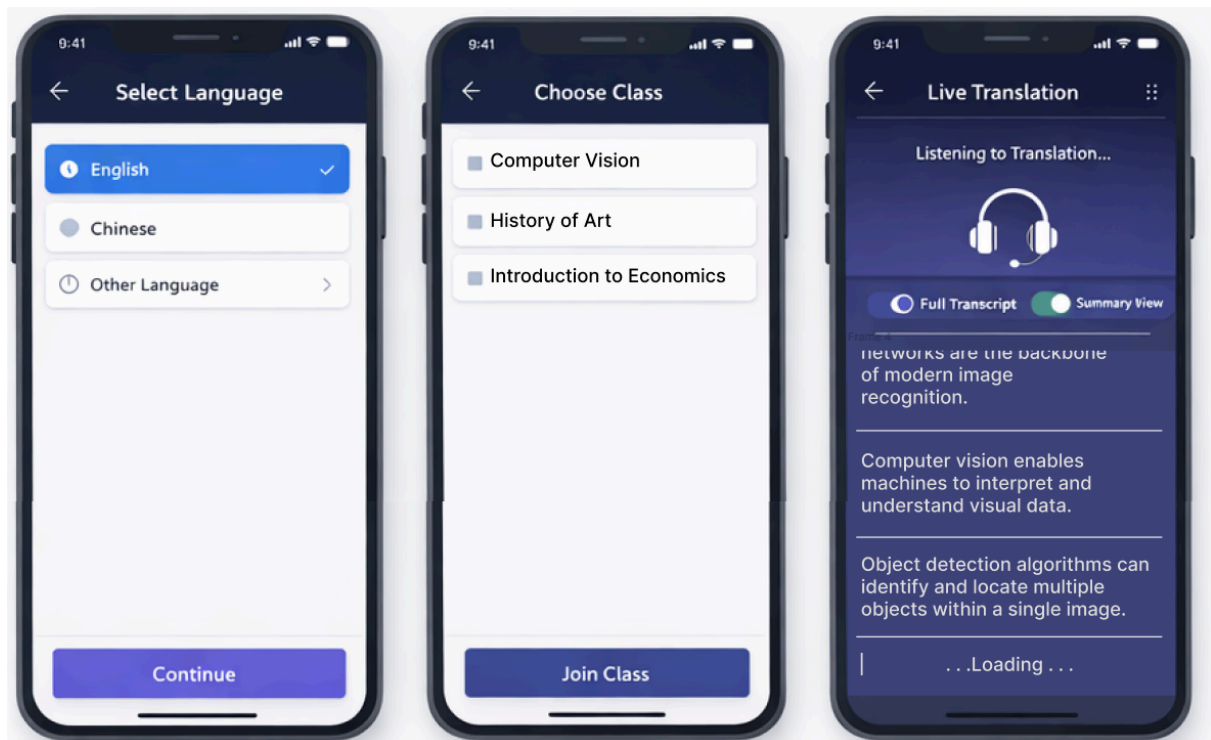
3. 인터페이스(UI) 구성 예시

교수님 화면



화면명	주요 구성 요소	비고
강의 대시보드	실시간 오디오 파형, 현재 진행 중인 자막 영역, RAG 참조 상태 표시	직관적인 스트리밍 UI

유학생 화면



화면명	주요 구성 요소	비고
설정/선택창	자막 언어 선택, 자막 모드(요약/전체), 강의 자료 업로드 버튼	사용자 맞춤형 설정
스크립트 뷰어	타임스탬프별 전체 대화 로그, 주요 전공 키워드 하이라이트	복습 및 검색 최적화

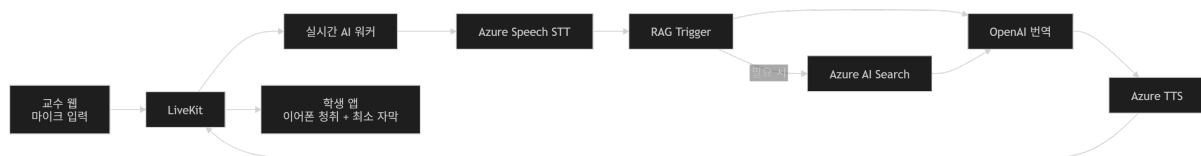
4. 기대 효과

- 유학생의 강의 이해도 향상
- 전공 용어 번역 정확도 개선 (RAG 기반)
- 교수-학생 간 커뮤니케이션 격차 감소
- 향후 다국어 확장 및 온라인 강의 적용 가능

5. MVP 이후

- Pinecone 기반 클라우드 확장
- 다국어 확대
- 웹/앱 통합 플랫폼

축약 버전



flowchart LR

A["교수 웹
마이크 입력"] --> B["LiveKit"]

B --> C["실시간 AI 워커"]

C --> D["Azure Speech STT"]

D --> E["RAG Trigger"]

E -->|필요 시| F["Azure AI Search"]

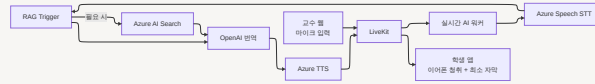
E --> G["OpenAI 번역"]

F --> G

G --> H["Azure TTS"]

H --> B

B --> I["학생 앱
이어폰 청취 + 최소 자막"]



수업 중 실시간 처리 흐름 다이어그램

```
sequenceDiagram
```

```
    autonumber
```

```
    participant Professor as 교수 웹
```

```
    participant LiveKit as LiveKit Room
```

```
    participant Worker as Session Orchestrator
```

```
    participant STT as Azure Speech STT
```

```
    participant Search as Azure AI Search
```

```
    participant OpenAI as OpenAI Responses API
```

```
    participant TTS as Azure Speech TTS
```

```
    participant Student as 학생 앱
```

```
    Professor->>LiveKit: professor.mic publish
```

```
    Worker->>LiveKit: professor.mic subscribe
```

```
    LiveKit->>Worker: 실시간 오디오 전달
```

```
    Worker->>STT: 오디오 스트림 전송
```

```
    STT-->>Worker: partial / final transcript
```

```
    Worker->>Worker: segment 안정화
```

```
    alt 일반 발화
```

```
        Worker->>OpenAI: 번역 요청 (RAG 없이)
```

```
    else 전공/애매한 발화
```

```
        Worker->>Search: 관련 강의자료 검색
```

```
        Search-->>Worker: context snippet 반환
```

```
        Worker->>OpenAI: context 포함 번역 요청
```

```
    end
```

```
    OpenAI-->>Worker: 언어별 번역 JSON
```

```
    par 중국어 음성 생성
```

```
        Worker->>TTS: zh-CN TTS 요청
```

```
        TTS-->>Worker: zh-CN audio
```

```
    and 베트남어 음성 생성
```

```
        Worker->>TTS: vi-VN TTS 요청
```

```
        TTS-->>Worker: vi-VN audio
```

```
    and 몽골어 음성 생성
```

```
        Worker->>TTS: mn-MN TTS 요청
```

```
        TTS-->>Worker: mn-MN audio
```

```
    end
```

```
    Worker->>LiveKit: tts.<locale> track publish
```

```
    Worker->>LiveKit: subtitle.<locale> stream publish
```

```
    LiveKit-->>Student: 선택 언어 오디오/자막 전달
```

